Mini Payment Gateway

Complete Build Roadmap - from your wallet API to a placement-ready flagship project
Java 17 + Spring Boot | PostgreSQL | Kafka | Redis | Docker | ~4 weeks, 3-4 hrs/day

---

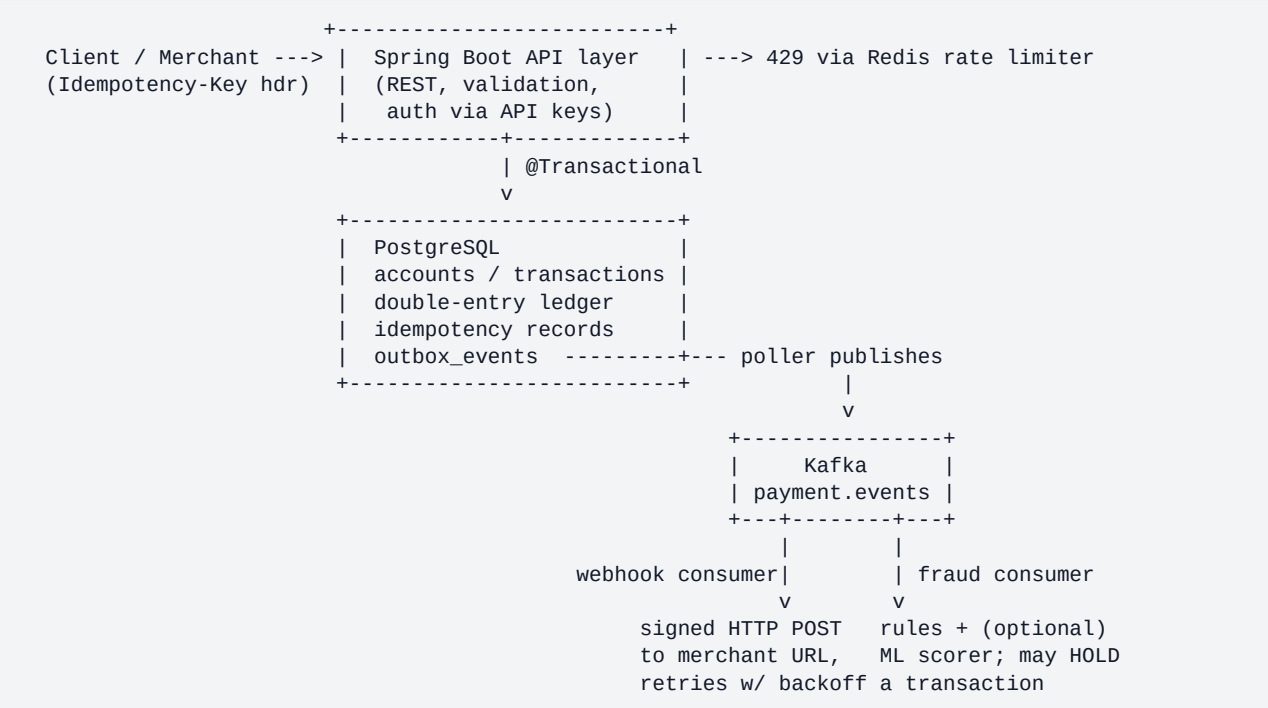
1. What you are building and why

A simplified payment gateway (think Razorpay / Stripe, minus real money movement): clients create payment requests through a REST API; the system moves value between accounts using a double-entry ledger, guarantees a payment is never executed twice even if the client retries, survives concurrent access without corrupting balances, notifies merchants asynchronously via signed webhooks, rate-limits abusive clients, and flags suspicious transactions through a fraud hook.

Why this project: it forces the exact concepts SDE interviewers at fintech-heavy IITD recruiters (Amex, Mastercard, Wells Fargo, Sprinklr, Razorpay-tier startups) probe hardest - ACID transactions, idempotency, race conditions, message queues, and API design. It also connects naturally to your AMEX Campus Challenge work through the fraud-scoring hook.

Your head start: you already know Java + Spring Boot and built a simple wallet transaction API. That wallet is version 0 of this system. Each phase below upgrades it one production concept at a time, so you are never starting from scratch.

2. Target architecture (end state)



3. Tech stack (final)

Layer	Choice	Why this choice
Language / framework	Java 17, Spring Boot 3 (Web, Data JPA, Validation, Actuator)	You already know it; dominant in Indian fintech; recruiters recognise it instantly.

Layer	Choice	Why this choice
Database	PostgreSQL 16 + Flyway migrations	Real ACID transactions and row locking - the heart of the project. Flyway = versioned schema, a professional habit.
Message broker	Apache Kafka (single broker via Docker)	Industry standard for payment events; unlocks outbox pattern, consumers, retries.
Cache / rate limit	Redis 7 + Bucket4j	Token-bucket rate limiting and velocity counters for fraud rules.
Testing	JUnit 5, Mockito, Testcontainers, k6	Testcontainers spins real Postgres/Kafka/Redis in tests; k6 gives the latency/TPS numbers for your resume.
Packaging / deploy	Docker + Docker Compose; Railway/Render or AWS EC2 free tier; GitHub Actions CI	One-command local spin-up; a live URL plus green CI badge on the README.

4. Timeline at a glance

Phase	Days	Deliverable
0. Setup and skeleton	1-2	Repo, Docker Compose (Postgres+Redis+Kafka), Spring Boot skeleton, Flyway baseline, health endpoint.
1. Core domain + double-entry ledger	3-7	Accounts, transactions, ledger entries; POST /payments moves money atomically; invariant: ledger always sums to zero.
2. Idempotency + concurrency	8-12	Idempotency-Key support (safe retries) and race-condition-proof balance updates, proven by a parallel-transfer test.
3. Async: outbox, Kafka, webhooks	13-18	Transactional outbox publishing payment events; webhook consumer with HMAC signatures and exponential-backoff retries.
4. Rate limiting + caching	19-21	Per-API-key token bucket in Redis returning 429 with headers; hot-read caching.
5. Fraud-flagging hook	22-24	Rules engine (velocity, amount, new-payee) producing a risk score; suspicious payments go to HELD. Optional ML scorer.
6. Testing + load testing	25-27	Unit + Testcontainers integration suite; k6 load test producing TPS and P50/P95/P99 latency numbers.
7. Deploy, CI/CD, polish	28-30	Live deployment, GitHub Actions pipeline, README with architecture diagram, demo GIF, resume bullets.

Rule of thumb: if a phase runs over by more than 2 days, cut scope inside the phase (the 'stretch' items), never the later phases - phases 2 and 3 carry the most interview value.

PHASE 0 — Setup and Project Skeleton

Days 1-2 | Goal: a running skeleton with one-command infrastructure

- Create a public GitHub repo **payment-gateway**. Add README stub, .gitignore (Java), MIT licence.
- Generate the project at start.spring.io: Maven, Java 17, dependencies - Spring Web, Spring Data JPA, Validation, Actuator, PostgreSQL Driver, Lombok, Spring for Apache Kafka, Spring Data Redis, Flyway.
- Write **docker-compose.yml** with postgres:16, redis:7, and a single-broker Kafka (bitnami/kafka or confluentinc images). Map ports; add volumes so data survives restarts.
- Add Flyway migration **V1__baseline.sql** (empty for now). Configure spring.datasource + spring.flyway in application.yml, with values overridable by environment variables (12-factor habit).
- Add a global exception handler (@RestControllerAdvice) returning a consistent JSON error shape - do this on day 1 and every later phase gets cleaner.
- Checkpoint: **docker compose up** then **./mvnw spring-boot:run**; GET /actuator/health returns UP.

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Spring Boot fundamentals refresh	Official guides: 'Building a RESTful Web Service', 'Accessing Data with JPA'	spring.io/guides
Project structure and layering	Amigoscode 'Spring Boot Full Course' (free, YouTube) - watch selectively for gaps	youtube.com/@amigoscode
Docker + Compose basics	'Get started' tutorial, then Compose docs	docs.docker.com/get-started
Flyway migrations	Flyway documentation - 'Why database migrations?' and quickstart	documentation.red-gate.com/flyway
Error handling pattern	Baeldung - search 'error handling for REST with Spring'	baeldung.com

PHASE 1 — Core Domain and the Double-Entry Ledger

Days 3-7 | Goal: money moves atomically; the ledger always sums to zero

This is the conceptual heart. Your wallet API almost certainly stored a single mutable *balance* column. Real payment systems instead record **immutable ledger entries** - every transaction writes a DEBIT row and a CREDIT row - and treat balance as derived (plus a cached copy for speed). This gives you an audit trail and a testable invariant: SUM(signed amounts) across the ledger is always zero.

Schema (Flyway V2__core.sql)

```
CREATE TABLE users (
  id            BIGSERIAL PRIMARY KEY,
  email         TEXT NOT NULL UNIQUE,
  api_key_hash  TEXT NOT NULL,            -- clients authenticate with an API key
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE accounts (
  id            BIGSERIAL PRIMARY KEY,
  user_id       BIGINT NOT NULL REFERENCES users(id),
  currency      CHAR(3) NOT NULL DEFAULT 'INR',
  balance       NUMERIC(18,2) NOT NULL DEFAULT 0,  -- cached; ledger is source of truth
  version       BIGINT NOT NULL DEFAULT 0,         -- for optimistic locking (Phase 2)
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT balance_non_negative CHECK (balance >= 0)
);

CREATE TYPE tx_status AS ENUM ('PENDING','COMPLETED','FAILED','HELD','REVERSED');

CREATE TABLE transactions (
  id               UUID PRIMARY KEY,
  idempotency_key  TEXT NOT NULL,
  source_account   BIGINT NOT NULL REFERENCES accounts(id),
  dest_account     BIGINT NOT NULL REFERENCES accounts(id),
  amount           NUMERIC(18,2) NOT NULL CHECK (amount > 0),
  currency         CHAR(3) NOT NULL,
  status           tx_status NOT NULL DEFAULT 'PENDING',
  failure_reason   TEXT,
  request_hash     TEXT NOT NULL,           -- detect key reuse with different body
  response_body    JSONB,                   -- replayed on idempotent retry
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_idem UNIQUE (idempotency_key)
);
CREATE INDEX idx_tx_source_created ON transactions (source_account, created_at DESC);

CREATE TYPE entry_dir AS ENUM ('DEBIT','CREDIT');

CREATE TABLE ledger_entries (
  id             BIGSERIAL PRIMARY KEY,
  transaction_id UUID   NOT NULL REFERENCES transactions(id),
  account_id     BIGINT NOT NULL REFERENCES accounts(id),
  direction      entry_dir NOT NULL,
  amount         NUMERIC(18,2) NOT NULL CHECK (amount > 0),
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_ledger_account ON ledger_entries (account_id, id);
```

API surface (model it on Stripe's API - interviewers notice)

```
POST /api/v1/accounts                     create account
POST /api/v1/accounts/{id}/topup          seed test money (dev only)
GET  /api/v1/accounts/{id}                cached balance + ledger-derived balance
GET  /api/v1/accounts/{id}/ledger?cursor= paginated statement (keyset pagination)
POST /api/v1/payments                     header: Idempotency-Key  body: {source,dest,amount,currency}
GET  /api/v1/payments/{id}                status + timeline
GET  /api/v1/payments?status=&cursor=     list, filterable
```

Build steps

- Entities + Spring Data repositories for the four tables. Use BigDecimal for money everywhere - never double (be ready to explain floating-point rounding in the interview).
- PaymentService.createPayment(): inside ONE @Transactional method - load accounts, check funds, insert transaction, insert both ledger entries, update both cached balances, mark COMPLETED. Any exception rolls back everything; that is atomicity, and you should be able to narrate it.
- DTOs with jakarta.validation (@NotNull, @Positive, currency pattern); never expose entities directly.
- Keyset (cursor) pagination on the ledger endpoint - say why OFFSET degrades on large tables.
- Checkpoint: manual test via Postman/curl - create 2 accounts, top up, pay, verify both balances AND that SELECT SUM(CASE WHEN direction='DEBIT' THEN -amount ELSE amount END) FROM ledger_entries = 0.
- Stretch: a GET /internal/reconcile endpoint comparing cached balances vs ledger-derived balances.

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Double-entry for developers	Modern Treasury 'Learn' series - 'Accounting for Developers' parts 1-2 (the single best primer)	moderntreasury.com/learn
Spring @Transactional semantics	Spring Framework docs, 'Transaction Management'; Baeldung 'transactional propagation isolation'	docs.spring.io / baeldung.com
Postgres transactions	PostgreSQL manual - tutorial chapter on transactions	postgresql.org/docs/current/tutorial-transactions.html
JPA pitfalls (N+1, mappings)	Vlad Mihalcea's blog - the reference for Hibernate performance	vladmihalcea.com
API design taste	Stripe API reference - copy their resource naming, error shape, pagination style	docs.stripe.com/api
Money handling	Baeldung - search 'BigDecimal money Java'	baeldung.com

PHASE 2 — Idempotency and Concurrency Control

Days 8-12 | Goal: retries never double-charge; races never corrupt balances

This phase alone can carry a 30-minute interview deep-dive. Two separate problems, two mechanisms:

2A. Idempotency keys (client retries)

- Contract: clients send a unique Idempotency-Key header per logical payment. Same key + same body = replay the stored response (200). Same key + different body (compare request_hash) = 422 error.
- Implementation: rely on the UNIQUE constraint, not a pre-check. Try INSERT of the transaction row first; catch DataIntegrityViolationException - fetch the existing row; if COMPLETED return its response_body, if PENDING return 409 'retry later'. A check-then-insert version has a race window - be able to explain why.
- Test: fire the same request twice sequentially (second returns identical body, no new ledger rows) and twice in parallel (exactly one performs the transfer).

2B. Concurrency control (simultaneous transfers)

- The bug you must create first: without locking, two parallel debits of 800 from an account holding 1000 can BOTH pass the funds check -> negative balance or lost update. Write a test with 50 threads hammering transfers between 3 accounts and watch it fail. Now fix it two ways and keep both in the repo:
- **Pessimistic:** SELECT ... FOR UPDATE on both account rows - always locked in ascending account-id order to prevent deadlock (classic interview follow-up: 'what if two transfers lock in opposite order?').

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select a from Account a where a.id in :ids order by a.id asc")
List<Account> lockAccounts(@Param("ids") List<Long> ids);
```

- **Optimistic:** @Version on Account; on OptimisticLockException retry up to N times with jitter. Trade-off to articulate: optimistic wins under low contention (no locks held), pessimistic wins when the same hot account is hit constantly (retry storms are worse). Make pessimistic the default; keep the optimistic path behind a config flag - having BOTH implemented is a rare, strong interview signal.
- Checkpoint: the 50-thread test passes; ledger sums to zero; no balance ever below zero; a short DECISIONS.md entry recording measured numbers for both strategies.

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Idempotency, definitive articles	Stripe blog 'Designing robust and predictable APIs with idempotency'; Brandur - 'Implementing Stripe-like idempotency keys in Postgres'	stripe.com/blog/idempotency ; brandur.org/idempotency-keys
Locking in JPA	Baeldung - 'pessimistic locking in JPA' and 'optimistic locking in JPA'	baeldung.com
Postgres row locks and deadlocks	PostgreSQL manual - 'Explicit Locking' chapter	postgresql.org/docs/current/explicit-locking.html
Isolation levels (theory)	Designing Data-Intensive Applications (Kleppmann), Chapter 7 'Transactions' - read this chapter fully this week	book / dataintensive.net
Video intuition	Hussein Nasser - videos on database locking, isolation levels, pooling	youtube.com/@hnasr

PHASE 3 — Async Processing - Outbox, Kafka, Webhooks

Days 13-18 | Goal: events leave the system reliably; merchants get signed, retried webhooks

Problem: after committing a payment you must notify the merchant. Publish-after-commit can crash between commit and publish (event lost); publish-inside-transaction can send an event for a rolled-back payment. The fix interviewers want to hear: the **transactional outbox pattern**.

Build steps

- Migration V3: **outbox_events**(id, aggregate_id, event_type, payload JSONB, status NEW/SENT, created_at) and **webhook_endpoints**(user_id, url, secret) and **webhook_deliveries**(event_id, attempt, status, next_retry_at, last_response_code).
- Inside the SAME transaction that completes a payment, insert an outbox row (payment.completed / payment.failed / payment.held). This is the whole trick: DB commit and 'event recorded' are atomic.
- A @Scheduled poller (every 500 ms) reads NEW outbox rows FOR UPDATE SKIP LOCKED, publishes to Kafka topic **payment.events** keyed by transaction id, marks SENT. SKIP LOCKED lets multiple pollers coexist - another nice interview morsel. (Mention Debezium/CDC as the production-grade alternative; polling is fine here.)
- Webhook consumer (@KafkaListener): loads merchant endpoint, POSTs the event JSON with header **X-Signature: HMAC-SHA256(secret, body)**, records the delivery. On non-2xx or timeout schedule retry with exponential backoff + jitter (1m, 5m, 25m... cap at N attempts -> DEAD status).
- Explain-ready: this pipeline is **at-least-once** - consumers may see duplicates, so the webhook receiver must be idempotent too (dedupe on event id). Exactly-once end-to-end is essentially a myth; saying that, calmly, is a senior-sounding answer.
- A tiny 'merchant simulator' Spring profile or Node script that receives webhooks, verifies the HMAC, and randomly fails 30 percent of the time - lets you demo retries live.
- Checkpoint: kill the app between DB commit and Kafka publish (breakpoint or System.exit in a test build); on restart the poller recovers and the event still goes out. That experiment IS the story you tell.
- Stretch: a dead-letter topic and a GET /internal/webhooks/failed admin view.

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Outbox pattern (canonical)	microservices.io - 'Transactional outbox'; Debezium blog 'Reliable microservices data exchange with the outbox pattern'	microservices.io ; debezium.io/blog
Kafka fundamentals	Confluent Developer free 'Kafka 101' course (topics, partitions, consumer groups, offsets)	developer.confluent.io
Spring Kafka	Spring for Apache Kafka reference - producer/consumer config, error handling, retries	docs.spring.io/spring-kafka/reference
Webhook design + HMAC	Stripe docs 'Webhooks' (signatures, retries, best practices) - mirror their design	docs.stripe.com/webhooks
Delivery semantics	DDIA Chapter 11 'Stream Processing' - at-least-once vs exactly-once discussion	book / dataintensive.net

PHASE 4 — Rate Limiting and Caching with Redis

Days 19-21 | Goal: abusive clients get 429s; hot reads stop hammering Postgres

- Token-bucket rate limiter per API key using **Bucket4j** backed by Redis (so the limit holds across multiple app instances - say that sentence in interviews). Example policy: 100 requests/min, burst 20.
- Implement as a servlet filter/interceptor running BEFORE controllers. On limit: HTTP 429 with headers X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After.
- Be ready to compare token bucket vs sliding window vs fixed window (fixed window's boundary-burst flaw is a favourite follow-up).
- Cache GET /accounts/{id} in Redis with a short TTL (5-10 s); evict/update on write (cache-aside). Articulate the staleness trade-off - and that you kept the ledger endpoint UNcached on purpose.
- Redis velocity counters you will reuse in Phase 5: INCR tx_count:{account}:{minute} with EXPIRE 60.
- Checkpoint: k6 smoke script proves 429s kick in at the configured threshold; cached read P95 visibly drops vs uncached (record both numbers).

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Rate limiting patterns	Redis glossary/blog on rate limiting; ByteByteGo 'Design a rate limiter' chapter (Alex Xu Vol 1)	redis.io ; bytebytego.com
Bucket4j usage	Bucket4j GitHub README + Baeldung 'rate limiting Spring Boot Bucket4j'	github.com/bucket4j/bucket4j
Spring Data Redis	Spring Data Redis reference; Baeldung 'Spring Boot cache with Redis'	docs.spring.io ; baeldung.com
Caching strategies	AWS/Redis writeups on cache-aside vs write-through; any of Arpit Bhayani's caching videos	youtube.com/@AsliEngineering

PHASE 5 — Fraud-Flagging Hook (your AMEX tie-in)

Days 22-24 | Goal: suspicious payments are HELD with an explainable risk score

- Rules engine first (deterministic, testable). Each rule returns a score: amount > account's 30-day P95; more than N payments in 60 s (Redis counter from Phase 4); first payment ever to this destination; round-amount + midnight-hours combo. Total >= threshold -> transaction status HELD, event payment.held emitted.
- Admin endpoints: GET /internal/held, POST /internal/transactions/{id}/release (moves HELD -> COMPLETED and resumes the normal event flow) - shows you thought about the human-in-the-loop.
- Run scoring synchronously inside the payment flow but with a strict time budget (e.g. 50 ms; on breach, default-allow and log). Latency-vs-risk trade-off = interview gold.
- **Optional ML extension (only if Phases 0-7 are done):** train a small gradient-boosted classifier on a public credit-card fraud dataset (Kaggle ULB dataset), export with ONNX, serve in-process via onnxruntime-java OR as a tiny FastAPI sidecar. Blend: final_score = 0.5*rules + 0.5*model. This one bullet links your AMEX story to a shipped system - exactly the bridge interviewers remember.
- Checkpoint: unit tests per rule; an integration test where a burst of payments triggers HELD; release flow works.

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Fraud features intuition	Stripe Radar technical guide (how real risk engines think about signals)	stripe.com/guides/radar
Dataset for optional ML	Kaggle - 'Credit Card Fraud Detection' (ULB) dataset	kaggle.com/datasets/mlg-ulb/creditcardfraud
ONNX in Java	ONNX Runtime Java API docs	onnxruntime.ai/docs
Imbalanced-data refresher	scikit-learn docs + your own AMEX notes (precision/recall vs accuracy - rehearse this)	scikit-learn.org

PHASE 6 — Testing and Load Testing

Days 25-27 | Goal: a green suite that proves correctness, and numbers that prove performance

Test pyramid for this project

- Unit: PaymentService funds check, fraud rules, HMAC signer, backoff calculator (Mockito for repos).
- Integration (Testcontainers - real Postgres, Kafka, Redis in disposable Docker containers): full payment happy path; idempotent replay; parallel duplicate key; 50-thread transfer storm asserting ledger-sum-zero and no negative balances; outbox recovery after simulated crash; webhook retry on 500.
- Keep the concurrency tests deterministic-ish: fixed thread pools + CountDownLatch to maximise collision probability; assert invariants, not exact interleavings.

Load testing with k6 (this produces your resume numbers)

```
// payments_load.js  - run: k6 run --vus 50 --duration 60s payments_load.js
import http from 'k6/http';
import { check } from 'k6';
import { uuidv4 } from 'https://jslib.k6.io/k6-utils/1.4.0/index.js';

export default function () {
  const res = http.post('http://localhost:8080/api/v1/payments',
    JSON.stringify({ sourceAccount: 1, destAccount: 2, amount: 10, currency: 'INR' }),
    { headers: { 'Content-Type': 'application/json',
                 'X-Api-Key': __ENV.KEY,
                 'Idempotency-Key': uuidv4() } });
  check(res, { 'status 200/201': r => r.status === 200 || r.status === 201 });
}
```

- Record: max sustainable TPS before P95 degrades, P50/P95/P99 latency, error rate - at 3 stages: (a) naive, (b) after locking strategy tuning, (c) with cache + rate limiter. The before/after deltas become your quantified bullets.
- Expose /actuator/metrics (+ optional Prometheus + Grafana docker-compose profile as a stretch - a dashboard screenshot in the README photographs extremely well).
- Checkpoint: CI runs the full suite; a LOAD_TEST.md file logs environment, commands, and results honestly (local laptop numbers are fine - just label them).

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Testcontainers	Testcontainers for Java - quickstart + Postgres/Kafka modules; Baeldung 'Testcontainers Spring Boot'	testcontainers.com
k6	Grafana k6 docs - 'Running k6', thresholds, results output	grafana.com/docs/k6
Spring Boot testing	Official 'Testing' chapter - @SpringBootTest, @DataJpaTest, slices	docs.spring.io/spring-boot
Metrics	Spring Boot Actuator + Micrometer docs	docs.spring.io/spring-boot

PHASE 7 — Deployment, CI/CD, and Polish

Days 28-30 | Goal: a live URL, a green pipeline, and a README that sells the work

- Multi-stage Dockerfile (Maven build stage -> slim JRE runtime). Compose file for the full stack.
- Deploy the app + Postgres + Redis on Railway or Render (simplest) or a single AWS EC2 free-tier box running compose (more impressive to ops-minded interviewers). Managed Kafka is overkill - on the cloud

box, run the single-broker container; document the difference.

- GitHub Actions: on push -> build, run unit + Testcontainers suite, build Docker image; on main -> deploy. Green badge in README.
- README structure: one-paragraph pitch; architecture diagram (draw in Excalidraw, export PNG); features table; 'design decisions' section (double-entry, outbox, locking trade-off - 5 lines each); load-test numbers; 'run locally in 2 commands'; demo GIF of a payment + webhook + a HELD fraud case.
- Write DECISIONS.md as you go, not at the end - it is your interview script.
- Final checkpoint: a stranger can clone, compose up, and make a payment in under 5 minutes.

Learn-when-you-need-it resources for this phase

Topic	Resource	Where
Dockerizing Spring Boot	Spring Boot official docs 'Container Images'; Baeldung 'dockerize Spring Boot'	docs.spring.io ; baeldung.com
GitHub Actions	Docs quickstart + 'Building and testing Java with Maven'	docs.github.com/actions
Deploy targets	Railway docs / Render docs (Docker deploys); AWS EC2 free-tier getting started	docs.railway.com ; render.com/docs

Resume bullets - fill the brackets with YOUR measured numbers

Format every bullet as Action + What + Stack + Measured result. Never claim a number you did not record.

```
Mini Payment Gateway | Java 17, Spring Boot, PostgreSQL, Kafka, Redis, Docker      [github.com/you/payment-gatew
- Built a payment processing system with a double-entry ledger in Spring Boot/PostgreSQL sustaining
  [X] TPS with P95 latency of [Y] ms under k6 load tests (50 VUs).
- Guaranteed exactly-once payment execution via idempotency keys and DB unique constraints; verified
  safe client retries under concurrent duplicate requests.
- Eliminated race conditions in concurrent transfers using ordered pessimistic row locking (benchmarked
  vs optimistic @Version retry: [A] vs [B] TPS under hot-account contention).
- Implemented transactional-outbox -> Kafka pipeline delivering HMAC-signed merchant webhooks with
  exponential-backoff retries; verified zero event loss across simulated crash-recovery tests.
- Added Redis token-bucket rate limiting ([N] req/min per API key) and cache-aside reads, cutting
  read P95 from [C] ms to [D] ms; rule-based fraud engine HELD [E]% of a synthetic attack burst.
```

Interview question bank - if you can answer these cold, the project has done its job

Area	Questions you must be able to answer
Ledger & transactions	Why double-entry instead of a balance column? What does @Transactional actually do? Which isolation level are you running and what anomalies does it allow? Why BigDecimal?
Idempotency	Walk through a duplicate request end-to-end. Why is check-then-insert racy? Same key, different body - what happens? How long do you retain keys?
Concurrency	Recreate the lost-update bug on a whiteboard. Why lock in ascending id order? Optimistic vs pessimistic - when does each win? What is SKIP LOCKED doing in your poller?
Async & Kafka	Why the outbox over publish-after-commit? At-least-once vs exactly-once - what does your system guarantee, and where must consumers dedupe? What does partition keying by transaction id buy you? What happens if the webhook endpoint is down for an hour?
Rate limiting / cache	Token bucket vs sliding window vs fixed window? Why does the limiter live in Redis and not app memory? What staleness does your cache tolerate and why is that acceptable?
Fraud	Why rules before ML? What is your latency budget and the fail-open decision? From AMEX: why is accuracy misleading on imbalanced data - which metric did you optimise?
Scale-up (design round)	How would this handle 10x traffic? (stateless app behind LB, read replicas, partitioned Kafka, hot-account mitigation via sharded sub-ledgers - know the words AND the trade-offs.)

Rules of engagement (read weekly)

- **Use AI tools to build fast, but own every line.** 2026 hiring rewards AI-assisted velocity - and the deep-dive is where it gets cashed in. If you cannot re-implement a piece on a whiteboard, rewrite it yourself.
- Small commits with real messages; the repo history is part of the artifact.
- If behind schedule: cut stretch items, then Phase 5's ML extension, then Grafana. Never cut Phases 2-3.
- After Day 30 this becomes background polish only - DSA is the main quest from August onward.
- Book pairing for the month: DDIA Ch. 7 during Phase 2, Ch. 11 during Phase 3; Alex Xu Vol 2 'Payment System' chapter after Phase 3 to see how your build maps to the canonical design answer.

End state, Day 30: a live URL, a green CI badge, a README with an architecture diagram and honest load-test numbers, and a DECISIONS.md you can narrate for 45 minutes. That is what 'internship-equivalent' looks like on a fresher resume.